

	SR UNIVERSITY	
	Campus Warangal	
	Program: II - B. Tech (CS& AI) Batch:	
	Name: Shivam Kumar Gautam	Reg. No: 2503A52457
	Department: Computer Science and AI	Semester: I
	AI Assisted Coding-Assignment	Date:

Q1 – AI-Assisted Bubble Sort Analysis

Prompt:-

Implement Bubble Sort in ascending order to numbers = [64, 34, 25, 12, 22, 11, 90]

and Count comparisons and swaps then Display the sorted list

now implement bubble sort and explain one complete pass of bubble sort after this count comparison and swaps and the test at least three input list and explain about an early exit optimisation and at last compare the original and improved version

Code:-

```
def bubble_sort_original(values):
```

```
    numbers = values.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    for pass_index in range(len(numbers) - 1):
```

```
        for index in range(len(numbers) - 1 - pass_index):
```

```
            comparisons += 1
```

```
            if numbers[index] > numbers[index + 1]:
```

```
                numbers[index], numbers[index + 1] = numbers[index + 1], numbers[index]
```

```
                swaps += 1
```

```
return numbers, comparisons, swaps
```

```
def bubble_sort_improved(values):
```

```
    numbers = values.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    for pass_index in range(len(numbers) - 1):
```

```
        swapped = False
```

```
        for index in range(len(numbers) - 1 - pass_index):
```

```
            comparisons += 1
```

```
            if numbers[index] > numbers[index + 1]:
```

```
                numbers[index], numbers[index + 1] = numbers[index + 1], numbers[index]
```

```
                swaps += 1
```

```
                swapped = True
```

```
        if not swapped:
```

```
            break
```

```
    return numbers, comparisons, swaps
```

```
def show_first_pass(values):
```

```
    numbers = values.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
for index in range(len(numbers) - 1):
    comparisons += 1
    if numbers[index] > numbers[index + 1]:
        numbers[index], numbers[index + 1] = numbers[index + 1], numbers[index]
        swaps += 1
```

```
print("One complete pass:")
print(" Before:", values)
print(" After: ", numbers)
print(" Comparisons:", comparisons)
print(" Swaps:", swaps)
```

```
test_lists = [
    [64, 34, 25, 12, 22, 11, 90],
    [1, 2, 3, 4, 5],
    [5, 4, 3, 2, 1],
]
```

```
show_first_pass(test_lists[0])
```

```
print("\nEarly-exit optimisation:")
```

```
print("The improved version stops when a complete pass makes no swaps, because the list is already sorted.")
```

```
for test_list in test_lists:
```

```
original_result = bubble_sort_original(test_list)
```

```
improved_result = bubble_sort_improved(test_list)
```

```
print("\nInput:", test_list)
```

```
print("Sorted list:", improved_result[0])
```

```
print("Original - comparisons:", original_result[1], "swaps:", original_result[2])
```

```
print("Improved - comparisons:", improved_result[1], "swaps:", improved_result[2])
```

```
print("\nComparison:")
```

```
print("The original always performs every possible pass.")
```

```
print("The improved version can use fewer comparisons on already or nearly sorted lists;  
swaps stay the same.")
```

output:-

One complete pass:

Before: [64, 34, 25, 12, 22, 11, 90]

After: [34, 25, 12, 22, 11, 64, 90]

Comparisons: 6

Swaps: 5

Early-exit optimisation:

The improved version stops when a complete pass makes no swaps, because the list is already sorted.

Input: [64, 34, 25, 12, 22, 11, 90]

Sorted list: [11, 12, 22, 25, 34, 64, 90]

Original - comparisons: 21 swaps: 14

Improved - comparisons: 21 swaps: 14

Input: [1, 2, 3, 4, 5]

Sorted list: [1, 2, 3, 4, 5]

Original - comparisons: 10 swaps: 0

Improved - comparisons: 4 swaps: 0

Input: [5, 4, 3, 2, 1]

Sorted list: [1, 2, 3, 4, 5]

Original - comparisons: 10 swaps: 10

Improved - comparisons: 10 swaps: 10

Comparison:

The original always performs every possible pass.

The improved version can use fewer comparisons on already or nearly sorted lists; swaps stay the same

Observation:-

The original Bubble Sort performs all passes, while the improved version uses early-exit optimization to reduce comparisons for already sorted or nearly sorted data

Q2 – Linear Search vs Binary Search

Prompt:-

Implement Linear and Binary Search to numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

Then Compare the number of comparisons

Now requesting you to implement linear search and binary search and explain why binary search requires sorted data after this count comparison for both algorithms and then compare the result and explain suitable use cases after this all suggest a valid optimization

Code:-

```
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]
```

```
def linear_search(arr, target):
```

```
    comparisons = 0
```

```
    for i in range(len(arr)):
```

```
        comparisons += 1
```

```
        if arr[i] == target:
```

```
            return i, comparisons
```

```
    return -1, comparisons
```

```
def binary_search(arr, target):
```

```
    comparisons = 0
```

```
    left = 0
```

```
    right = len(arr) - 1
```

```
    while left <= right:
```

```
        comparisons += 1
```

```
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
```

```
            return mid, comparisons
```

```
elif arr[mid] < target:
```

```
    left = mid + 1
```

```
else:
```

```
    right = mid - 1
```

```
return -1, comparisons
```

```
search_value = 45
```

```
linear_result, linear_comparisons = linear_search(numbers, search_value)
```

```
binary_result, binary_comparisons = binary_search(numbers, search_value)
```

```
print("Linear Search:")
```

```
if linear_result != -1:
```

```
    print(f"Element {search_value} found at index {linear_result} in {linear_comparisons} comparisons.")
```

```
else:
```

```
    print(f"Element {search_value} not found in {linear_comparisons} comparisons.")
```

```
print("\nBinary Search:")
```

```
if binary_result != -1:
```

```
    print(f"Element {search_value} found at index {binary_result} in {binary_comparisons} comparisons.")
```

```
else:
```

```
    print(f"Element {search_value} not found in {binary_comparisons} comparisons.")
```

```
print("\nWhy binary search needs sorted data:")
```

```
print("Binary search works by comparing the target with the middle element and then discarding half the list.")
```

```
print("This only works correctly if the data is sorted, because the left half must contain values smaller than the middle value and the right half larger values.")
```

```
print("If the list is unsorted, the middle value does not tell us which half can contain the target.")
```

```
print("\nComparison result:")
```

```
print(f"Linear search uses {linear_comparisons} comparisons.")
```

```
print(f"Binary search uses {binary_comparisons} comparisons.")
```

```
print("Binary search is more efficient because it removes half of the remaining elements each time.")
```

```
print("\nSuitable use cases:")
```

```
print("- Linear search: good for small or unsorted lists, or when data is rarely searched and simplicity matters.")
```

```
print("- Binary search: best for large sorted datasets, such as a dictionary, phonebook, or sorted database index.")
```

```
print("\nValid optimization:")
```

```
print("If the data is searched many times, sort it once and then use binary search instead of repeated linear searches.")
```

```
print("This reduces the search cost from  $O(n)$  per lookup to  $O(\log n)$  per lookup after sorting, which is a strong optimization for large datasets.")
```

output:-

Linear Search:

Element 45 found at index 5 in 6 comparisons.

Binary Search:

Element 45 found at index 5 in 3 comparisons.

Why binary search needs sorted data:

Binary search works by comparing the target with the middle element and then discarding half the list.

This only works correctly if the data is sorted, because the left half must contain values smaller than the middle value and the right half larger values.

If the list is unsorted, the middle value does not tell us which half can contain the target.

Comparison result:

Linear search uses 6 comparisons.

Binary search uses 3 comparisons.

Binary search is more efficient because it removes half of the remaining elements each time.

Suitable use cases:

- Linear search: good for small or unsorted lists, or when data is rarely searched and simplicity matters.
- Binary search: best for large sorted datasets, such as a dictionary, phonebook, or sorted database index.

Valid optimization:

If the data is searched many times, sort it once and then use binary search instead of repeated linear searches.

This reduces the search cost from $O(n)$ per lookup to $O(\log n)$ per lookup after sorting, which is a strong optimization for large datasets

Observation:-

Binary Search requires sorted data and performs fewer comparisons than Linear Search for the tested input, making it more efficient for large sorted datasets.

Q3 – Compare Sorting Algorithms

Prompt:-

Sort using Bubble Sort, Selection Sort and Insertion Sort to data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56] and Compare their performance

implement all three sorting algorithms, use the same input for each algorithm then count comparisons and swaps/shifts and after this explain strengths and weaknesses and which algorithm suits nearly sorted data at last Compare experimental results with theoretical complexity

code:-

```
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]
```

```
def bubble_sort(values):
```

```
    result = values.copy()
```

```
    comparisons = 0
```

```
    swaps = 0
```

```
    for pass_index in range(len(result) - 1):
```

```
        swapped = False
```

```
        for index in range(len(result) - 1 - pass_index):
```

```
            comparisons += 1
```

```
            if result[index] > result[index + 1]:
```

```
                result[index], result[index + 1] = result[index + 1], result[index]
```

```
                swaps += 1
```

```
        swapped = True
    if not swapped:
        break
    return result, comparisons, swaps
```

```
def selection_sort(values):
    result = values.copy()
    comparisons = 0
    swaps = 0
    for index in range(len(result) - 1):
        smallest_index = index
        for search_index in range(index + 1, len(result)):
            comparisons += 1
            if result[search_index] < result[smallest_index]:
                smallest_index = search_index
        if smallest_index != index:
            result[index], result[smallest_index] = result[smallest_index], result[index]
            swaps += 1
    return result, comparisons, swaps
```

```
def insertion_sort(values):
    result = values.copy()
    comparisons = 0
    shifts = 0
    for index in range(1, len(result)):
        current_value = result[index]
```

```

search_index = index - 1
while search_index >= 0:
    comparisons += 1
    if result[search_index] <= current_value:
        break
    result[search_index + 1] = result[search_index]
    shifts += 1
    search_index -= 1
    result[search_index + 1] = current_value
return result, comparisons, shifts

print("Input data:", data)
print("\nExperimental results (same input for every algorithm):")

for name, sort_function, movement_name in [
    ("Bubble Sort", bubble_sort, "swaps"),
    ("Selection Sort", selection_sort, "swaps"),
    ("Insertion Sort", insertion_sort, "shifts"),
]:
    sorted_data, comparisons, movements = sort_function(data)
    print(f"{name}: {sorted_data}")
    print(f"  Comparisons: {comparisons}, {movement_name}: {movements}")

print("\nStrengths and weaknesses:")
print("- Bubble Sort: simple and adaptive with an early-exit check, but can do many swaps and is  $O(n^2)$  in the average and worst cases.")

```

```
print("- Selection Sort: uses very few swaps and O(1) extra space, but always makes O(n^2) comparisons and is not adaptive.")
```

```
print("- Insertion Sort: simple, stable, and fast for nearly sorted data, but shifts many values on reversed or random input.")
```

```
print("\nBest choice for nearly sorted data: Insertion Sort, because its best case is O(n) when only a few shifts are needed.")
```

```
print("\nTheoretical versus experimental complexity:")
```

```
print("- Bubble Sort: best O(n) with early exit; average/worst O(n^2).")
```

```
print("- Selection Sort: best/average/worst O(n^2), regardless of input order.")
```

```
print("- Insertion Sort: best O(n); average/worst O(n^2).")
```

```
print("The measured counts are for one input of length 10, so they illustrate the theory but do not replace tests on larger or differently ordered data.")
```

output:-

Input data: [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

Experimental results (same input for every algorithm):

Bubble Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 44, swaps: 24

Selection Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 45, swaps: 6

Insertion Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]

Comparisons: 31, shifts: 24

Strengths and weaknesses:

- Bubble Sort: simple and adaptive with an early-exit check, but can do many swaps and is $O(n^2)$ in the average and worst cases.
- Selection Sort: uses very few swaps and $O(1)$ extra space, but always makes $O(n^2)$ comparisons and is not adaptive.
- Insertion Sort: simple, stable, and fast for nearly sorted data, but shifts many values on reversed or random input.

Best choice for nearly sorted data: Insertion Sort, because its best case is $O(n)$ when only a few shifts are needed.

Theoretical versus experimental complexity:

- Bubble Sort: best $O(n)$ with early exit; average/worst $O(n^2)$.
- Selection Sort: best/average/worst $O(n^2)$, regardless of input order.
- Insertion Sort: best $O(n)$; average/worst $O(n^2)$.

The measured counts are for one input of length 10, so they illustrate the theory but do not replace tests on larger or differently ordered data.

Observation:-

Bubble Sort, Selection Sort, and Insertion Sort produce the same sorted result, but their comparison and movement counts differ; Insertion Sort is particularly suitable for nearly sorted data.

Q4 – Optimising a Student Search and Sort Program

Prompt:-

```
students = [ ("Arun", 82), ("Bala", 65), ("Chitra", 91), ("Divya", 74), ("Esha", 88)]
```

Write functions for highest mark, sorting and name search and Create a straightforward solution first now identify unnecessary operations, provide more efficient approach and then Implement only suggestions you understand after this all Compare original and improved time complexity

Code:-

```
students = [  
    ("Arun", 82),  
    ("Bala", 65),  
    ("Chitra", 91),  
    ("Divya", 74),  
    ("Esha", 88),  
]
```

```
def highest_mark(student_list):  
    """Return the student with the highest mark."""  
    return max(student_list, key=lambda student: student[1])
```

```
def sort_by_mark(student_list):  
    """Return students sorted from highest mark to lowest mark."""  
    return sorted(student_list, key=lambda student: student[1], reverse=True)
```

```
def find_student(student_list, name):  
    """Straightforward search: scan the list until a name matches."""  
    return next(  
        (student for student in student_list if student[0].casefold() == name.casefold()),  
        None,  
    )
```

```
def build_student_index(student_list):  
    """Build a case-insensitive name index for efficient repeated searches."""  
    return {name.casefold(): (name, mark) for name, mark in student_list}
```

```

def find_student_fast(student_index, name):
    """Search the pre-built index in constant average time."""
    return student_index.get(name.casefold())

# The straightforward approach is clear and works well for a small list.
top_student = highest_mark(students)
sorted_students = sort_by_mark(students)
search_name = "Chitra"
result = find_student(students, search_name)

print(f"Highest mark: {top_student[0]} - {top_student[1]}")

print("Students sorted by mark:")
for name, mark in sorted_students:
    print(f"{name}: {mark}")

if result:
    print(f"Search result: {result[0]} - {result[1]}")
else:
    print(f"{search_name} was not found.")

# Improvement: casefold each stored name once, then reuse the index.
student_index = build_student_index(students)
fast_result = find_student_fast(student_index, search_name)
print(f"Fast search result: {fast_result[0]} - {fast_result[1]}")

```

""""

Time-complexity comparison:

Operation	Straightforward approach	Improved approach
Highest mark	$O(n)$	$O(n)$
Sorting	$O(n \log n)$	$O(n \log n)$
One name search	$O(n)$	$O(1)$ average
Building an index	Not required	$O(n)$

The index is worthwhile when there are many name searches. It avoids repeated list scans and repeated case conversion. Sorting cannot become $O(n)$ here because every student must be ordered by mark.

""""

Output:-

Highest mark: Chitra - 91

Students sorted by mark:

Chitra: 91

Esha: 88

Arun: 82

Divya: 74

Bala: 65

Search result: Chitra - 91

Fast search result: Chitra – 91

Observation :-

Building a name index improves repeated student searches from **$O(n)$** to **$O(1)$ average**, while finding the highest mark and sorting retain their original time complexities

Q5 – Optimisation Challenge: Best Combination

Prompt:-

```
items = [ ("Laptop", 60000),("Phone", 30000),("Tablet", 20000), ("Watch",  
10000),("Headphones", 5000)]
```

```
budget = 50000
```

Find a combination of items whose total price

is as high as possible without exceeding the budget

Design a solution to maximise total value without exceeding the budget then start with a simple approach and explain it, to compare brute force, greedy and dynamic programming then determine which approach guarantees an optimal solution for this formulation after this Implement the selected solution test at least three budgets and Explain time and space complexity and possible optimisation for larger inputs

Code:-

```
from itertools import combinations
```

```
items = [  
    ("Laptop", 60000),  
    ("Phone", 30000),  
    ("Tablet", 20000),  
    ("Watch", 10000),  
    ("Headphones", 5000),  
]
```

```
def brute_force_knapsack(items_list, budget):
```

```
    """Simple approach: try every subset of items and keep the best valid one."""
```

```

best_combo = []
best_value = 0

for r in range(1, len(items_list) + 1):
    for combo in combinations(items_list, r):
        total_cost = sum(cost for _, cost in combo)
        total_value = sum(value for _, value in combo)
        if total_cost <= budget and total_value > best_value:
            best_combo = list(combo)
            best_value = total_value

return best_combo, best_value

```

```

def greedy_knapsack(items_list, budget):
    """Greedy idea: select items by highest value/cost first."""
    remaining_budget = budget
    chosen = []
    for name, cost in sorted(items_list, key=lambda x: x[1], reverse=True):
        if cost <= remaining_budget:
            chosen.append((name, cost))
            remaining_budget -= cost
    return chosen, sum(cost for _, cost in chosen)

```

```

def dynamic_programming_knapsack(items_list, budget):
    """Guaranteed optimal 0/1 knapsack using dynamic programming."""
    n = len(items_list)

```

```
dp = [[0] * (budget + 1) for _ in range(n + 1)]
keep = [[None] * (budget + 1) for _ in range(n + 1)]
```

```
for i in range(1, n + 1):
    name, cost = items_list[i - 1]
    for cap in range(budget + 1):
        dp[i][cap] = dp[i - 1][cap]
        keep[i][cap] = None
        if cost <= cap:
            candidate = dp[i - 1][cap - cost] + cost
            if candidate > dp[i][cap]:
                dp[i][cap] = candidate
                keep[i][cap] = (i - 1, cap - cost)
```

```
chosen = []
cap = budget
for i in range(n, 0, -1):
    prev = keep[i][cap]
    if prev is not None:
        idx, new_cap = prev
        chosen.append(items_list[idx])
        cap = new_cap
```

```
chosen.reverse()
return chosen, dp[n][budget]
```

```

print("Comparing approaches for the same budget problem")

print("=" * 60)

for budget in [50000, 35000, 25000]:

    brute_combo, brute_value = brute_force_knapsack(items, budget)

    greedy_combo, greedy_value = greedy_knapsack(items, budget)

    dp_combo, dp_value = dynamic_programming_knapsack(items, budget)

    print(f"\nBudget: {budget}")

    print(f"Brute force: {[ (name, cost) for name, cost in brute_combo]} -> {brute_value}")

    print(f"Greedy:   {[ (name, cost) for name, cost in greedy_combo]} -> {greedy_value}")

    print(f"Dynamic:  {[ (name, cost) for name, cost in dp_combo]} -> {dp_value}")

    if brute_value == dp_value:

        print("Optimal result confirmed by brute force and dynamic programming.")

    else:

        print("Warning: dynamic programming differs from brute force, which should not happen.")

    if greedy_value < dp_value:

        print("Greedy is not optimal for this instance.")

print("\nConclusion:")

print("- Brute force checks every subset and guarantees the best answer, but it is exponential.")

print("- Greedy is fast, but it may choose a locally attractive item and miss the better overall combination.")

```

```
print("- Dynamic programming guarantees the optimal solution for the 0/1 knapsack  
formulation in polynomial time.")
```

```
print("- Therefore, the dynamic-programming approach is the correct choice for this  
problem.")
```

output:-

Comparing approaches for the same budget problem

=====

Budget: 50000

Brute force: [('Phone', 30000), ('Tablet', 20000)] -> 50000

Greedy: [('Phone', 30000), ('Tablet', 20000)] -> 50000

Dynamic: [('Phone', 30000), ('Tablet', 20000)] -> 50000

Optimal result confirmed by brute force and dynamic programming.

Budget: 35000

Brute force: [('Phone', 30000), ('Headphones', 5000)] -> 35000

Greedy: [('Phone', 30000), ('Headphones', 5000)] -> 35000

Dynamic: [('Phone', 30000), ('Headphones', 5000)] -> 35000

Optimal result confirmed by brute force and dynamic programming.

Budget: 25000

Brute force: [('Tablet', 20000), ('Headphones', 5000)] -> 25000

Greedy: [('Tablet', 20000), ('Headphones', 5000)] -> 25000

Dynamic: [('Tablet', 20000), ('Headphones', 5000)] -> 25000

Optimal result confirmed by brute force and dynamic programming.

Conclusion:

- Brute force checks every subset and guarantees the best answer, but it is exponential.
- Greedy is fast, but it may choose a locally attractive item and miss the better overall combination.
- Dynamic programming guarantees the optimal solution for the 0/1 knapsack formulation in polynomial time.
- Therefore, the dynamic-programming approach is the correct choice for this problem.

Observation :-

Brute Force and Dynamic Programming guarantee the optimal solution for the given 0/1 knapsack problem, while Greedy is faster but does not always guarantee an optimal solution.